

图像内容感知缩放实验报告

Seam Carving

PB23000052 杨硕

实验目的

本次实验实现一种内容感知的图像缩放方法 Seam Carving。与普通缩放直接对整幅图像做统一拉伸不同，Seam Carving 通过寻找图像中能量较低的连通路径，并对这些路径进行删除或插入，从而在改变图像宽高的同时尽量保留主体结构。实验目标是理解 Seam Carving 的基本原理，并完成 Python 版本的交互式实现。

本实验的主要代码位于 `hw_1/op_1/code_template/seam_carving.py`。程序读取输入图像后，提供水平和垂直两个方向的缩放滑块，并在点击按钮后输出重定向结果。

算法原理

Seam Carving

Seam Carving 由 Avidan 和 Shamir 在 2007 年提出，其核心思想是用一条从图像一侧连接到另一侧的低能量路径作为最小代价删减单元。对于垂直 seam，路径从图像顶端到底端，每一行恰好经过一个像素，且相邻两行对应像素的横坐标之差不超过 1。若第 i 行所经过的列坐标为 $x(i)$ ，则有

$$|x(i) - x(i-1)| \leq 1.$$

水平 seam 的定义与之类似，只是路径从左到右穿过图像。实际实现时可以通过转置图像复用垂直 seam 的算法，从而避免重复写两套逻辑。

Seam Carving 的关键在于两个问题：如何定义像素重要性，以及如何高效求出最小总代价的 seam。前者由能量函数决定，后者由动态规划完成。

能量函数

论文中常用梯度幅值作为像素能量，而本实验按照作业要求采用 Laplacian 形式的能量函数。对图像的每个颜色通道分别计算二阶差分响应，再将三个通道响应的平方相加，得到最终能量图：

$$e(x, y) = \sum_{c \in \{R, G, B\}} \left(\nabla^2 I_{c(x, y)} \right)^2.$$

实现时使用的离散卷积核为

$$K = \begin{pmatrix} 0.5 & 1 & 0.5 \\ 1 & -6 & 1 \\ 0.5 & 1 & 0.5 \end{pmatrix}.$$

能量较高的位置通常对应边缘、轮廓或纹理明显的区域，而平坦背景的能量较低，更容易被 seam carving 删除或复制。

最优 Seam 的动态规划

设 $M(i, j)$ 表示从图像第一行走到位置 (i, j) 的最小累计能量，则垂直 seam 的动态规划转移满足

$$M(i, j) = e(i, j) + \min(M(i-1, j-1), M(i-1, j), M(i-1, j+1)).$$

初始条件为第一行

$$M(0, j) = e(0, j).$$

在计算完全部累计能量后，从最后一行中选取最小值作为 seam 终点，再利用前驱数组逐行回溯，就可以恢复整条最优路径。

算法实现

能量图计算

函数 `compute_energy(im)` 用于生成输入图像的二维能量图。代码先将图像转为 `float32`，以减少计算时的内存压力；然后对每个通道分别做边界填充，再利用 3×3 Laplacian 核计算响应，最后把三个通道的响应平方和累加到总能量图中。

这一实现对应的意义是：图像中边界和纹理越明显的位置，Laplacian 响应越大，因此 seam 更倾向于绕开这些区域。对于海面、天空这类平滑区域，响应通常较低，算法会优先从这些地方删除或插入像素路径。

最优垂直 Seam 搜索

函数 `find_vertical_seam(energy)` 实现垂直方向最优 seam 的搜索。代码中维护两个矩阵：

- `cost`：当前位置的最小累计代价。
- `parent`：当前像素来自上一行的哪一列。

对于每一行，程序比较上一行左、中、右三个候选位置的累计代价，并把最小者记录下来。处理完成后，从最后一行代价最小的位置开始向上回溯，即可得到整条 seam。

这一部分本质上是动态规划，只不过在实现上没有对每个像素再套一层 Python 循环，而是采用逐行向量化更新，以降低交互式界面中的等待时间。

Seam 删除

函数 `remove_vertical_seam(im, seam)` 根据 seam 中记录的列坐标，在每一行删除一个像素，从而将图像宽度减少 1。实现方法是构造一个与图像宽高一致的布尔掩码，把 seam 对应位置标记为 `False`，再利用布尔索引一次性取出剩余像素，并 `reshape` 为新的图像尺寸。

这种方式比逐行拼接数组更紧凑，也更适合 `numpy` 的向量化数据布局。

Seam 插入

对于图像放大，若每次只找到一条最优 seam 并立即插入，算法往往会不断重复利用同一片低能量区域，导致平坦区域被明显拉伸。为此，本实验采用了更接近原论文的做法。

函数 `collect_vertical_seams(im, k)` 会先在临时图像上连续删除前 k 条最优 seam，并用 `index_map` 记录这些 seam 在原图中的列坐标。随后，函数 `insert_vertical_seams(im, seams)` 再按照这些原始坐标批量把 seam 插回图像。插入时，新像素由当前像素与右邻居像素平均得到，这样既能扩大图像尺寸，又能减轻反复复制同一 seam 带来的伪影。

考虑到一次性放大过多会产生明显失真，代码中的 `enlarge_width(im, target_w, max_step_ratio=1.5)` 采用分步放大策略，每一步最多放大到当前宽度的 1.5 倍，再重复收集和插入 seam，直到达到目标尺寸。

双方向缩放控制

总控函数 `seam_carve_image(im, sz)` 负责处理目标大小 (`target_h`, `target_w`)。其流程可以概括为:

1. 若当前宽度大于目标宽度, 则重复执行“计算能量图、寻找最优垂直 seam、删除 seam”。
2. 若当前高度大于目标高度, 则先转置图像, 再复用垂直 seam 删除逻辑。
3. 若当前宽度小于目标宽度, 则调用 `enlarge_width` 做批量 seam 插入。
4. 若当前高度小于目标高度, 则同样通过转置后复用宽度放大逻辑。

对应伪代码如下:

```
function seam_carve_image(im, target_h, target_w):
    out <- float32(im)

    while width(out) > target_w:
        seam <- find_vertical_seam(compute_energy(out))
        out <- remove_vertical_seam(out, seam)

    while height(out) > target_h:
        out <- transpose(out)
        seam <- find_vertical_seam(compute_energy(out))
        out <- remove_vertical_seam(out, seam)
        out <- transpose(out)

    if width(out) < target_w:
        out <- enlarge_width(out, target_w)

    if height(out) < target_h:
        out <- transpose(out)
        out <- enlarge_width(out, target_h)
        out <- transpose(out)

    return uint8(clip(out, 0, 255))
```

实验结果

缩小与放大结果

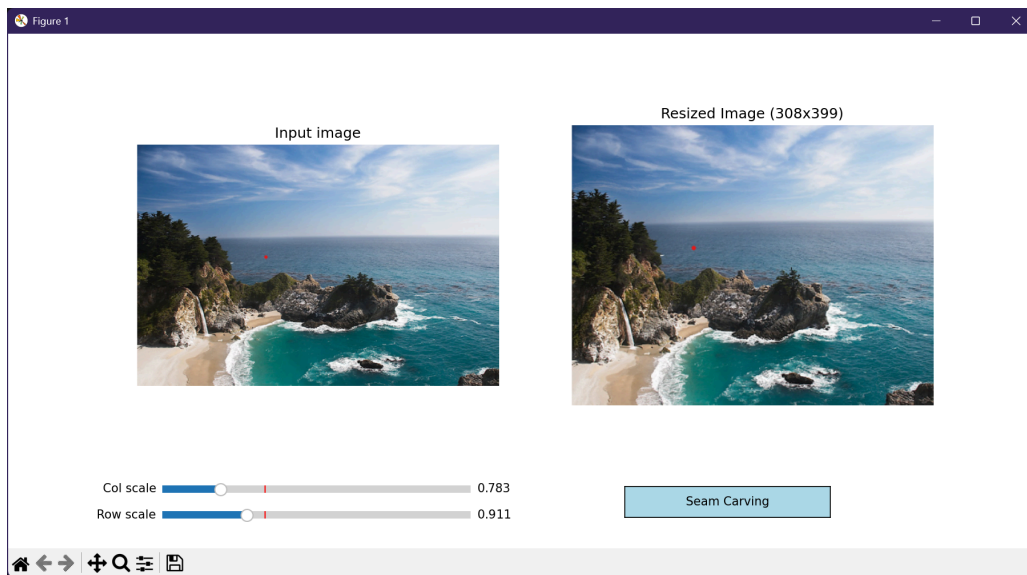


Figure 1: 图像缩小结果。可以看到算法优先删除低能量区域中的 seam，在压缩宽高时尽量保留了海岸、礁石与海面的主要结构关系。

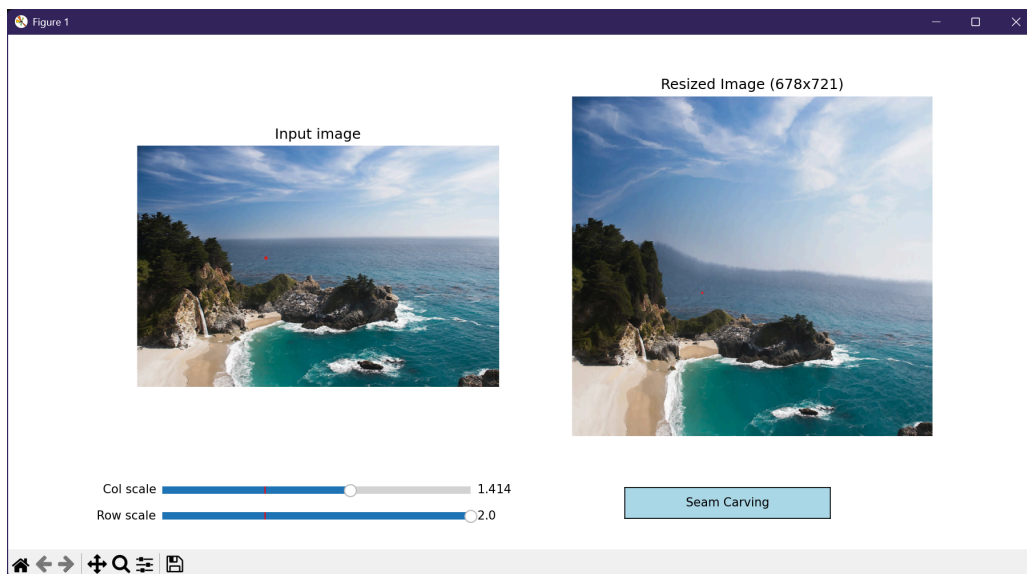


Figure 2: 图像放大结果。程序先收集多条待插入 seam 再统一插入，相比逐条立即插入的方法，能够减轻大面积平坦区域被重复拉伸的问题。

结果分析

实验结果表明，在图像缩小时，Seam Carving 通常能够比普通缩放更好地保留主要景物结构。例如海岸、岩石和海浪边界不会被平均压扁，而是尽量维持原有的视觉关系。这说明按能量函数选择删除路径的方法确实能够在一定程度上实现内容感知缩放。

在图像放大时，若直接逐条寻找并立即插入 seam，天空或海面这类平坦区域很容易被重复复制，形成大块拉伸伪影。因此本实验改为先收集一组待插入的 seam，再统一插入，结果相较最基础的放大策略有明显改善。不过，当放大比例过大时，局部不自然现象仍然存在，这也是 seam insertion 方法本身的局限。

总结

本实验完成了一个基于 Seam Carving 的图像内容感知缩放程序，并实现了以下关键部分：

- 基于 Laplacian 的像素能量函数；
- 基于动态规划的最优 seam 搜索；
- 通过 seam 删除实现图像缩小；
- 通过批量 seam 插入实现图像放大；
- 通过图像转置统一处理水平与垂直两个方向。

总体来看，Seam Carving 相比普通缩放更能保留图像中的主体内容，但代价是计算量较大。由于每删除或插入一条 seam 都需要重新计算整张图的能量图 and 最优路径，因此在交互式界面中，当目标尺寸变化较大时会出现明显等待时间。后续若要进一步提升实用性，可以考虑能量图局部更新、更高效的底层实现，或在图形界面中加入低分辨率预览机制。